Chapter 1 Basics concepts of hydrological models: routing

Olivier Bonte

2026-07-02

To get more insight into the routing concepts explained in the theory, a computational guide is given here, leveraging both symbolic and numerical computing.

1 Reservoir cascade models

1.1 Cascade of linear reservoirs

1.1.1 First order linear reservoir with constant input

A linear reservoir with storage S [m^3] and a constant input I [m^3/s] is characterised by

$$S = KQ \tag{1}$$

with K [s] the residence time and Q [m^3/s] the outflow. The mass balance equation is given by

$$\frac{dS}{dt} = K\frac{dQ}{dt} = I - Q \tag{2}$$

As this is a linear ordinary differential equation, it can be solved analytically using Laplace transforms (see your course in the 2nd bachelor year on Differential Equations). Remember that although the definition of a Laplace transform might seem complex

$$F(s) = \mathcal{L}\{f(t)\} = \int_0^\infty f(t)e^{-st}dt,$$

in practice you only need to apply a limited set of transforms to solve linear ODEs, which are summarised in the table below.

$f(t)$	$F(s) = \mathcal{L}\{f(t)\}$
$H(t)$	$\frac{1}{s}$
t^n	$\frac{n!}{s^{n+1}}$
e^{at}	$\frac{1}{s-a}$
$f'(t)$	$sF(s) - f(0)$

where $H(t)$ is the Heaviside function, which is defined as $H(t) = 0$ for $t < 0$ and $H(t) = 1$ for $t \geq 0$. n is a positive integer and a is a real number. Applying the Laplace transform to equation Equation 2 with $Q(t=0) = Q_0 = S_0/K$ and $q(s) = \mathcal{L}\{Q(t)\}$ gives:

$$K(sq(s) - Q_0) = \frac{I}{s} - q(s)$$

Isolating $q(s)$ gives:

$$q(s) = \frac{I}{s(1+Ks)} + \frac{KQ_0}{1+Ks} \tag{3}$$

We now need to apply the inverse Laplace transform to get $Q(t)$. Trivially, $\mathcal{L}^{-1}(q(s)) = Q(t)$. For the term containing Q_0:

$$\mathcal{L}^{-1}\left(\frac{KQ_0}{1+Ks}\right) = \frac{K}{K} Q_0 \mathcal{L}^{-1}\left(\frac{1}{1/K + s}\right) = Q_0 e^{-t/K} \tag{4}$$

The term with I is a bit more convolved, but can be solved using partial fraction decomposition:

$$\frac{I}{s(1+Ks)} = \frac{A}{s} + \frac{B}{1+Ks}$$

To solve for A, multiply both sides by s and set $s = 0$:

$$\left.\frac{I}{1+Ks}\right|_{s=0} = A \iff A = I$$

To solve for B, multiply both sides by $(1+Ks)$ and set $s = -1/K$:

$$\left.\frac{I}{s}\right|_{s=-1/K} = B \iff B = -IK$$

Now we can apply the inverse laplace transform on the partial fraction decomposition:

$$\mathcal{L}^{-1}\left(\frac{I}{s} - \frac{IK}{1+Ks}\right) = \mathcal{L}^{-1}\left(\frac{I}{s} - \frac{I}{s+1/K}\right) = I - Ie^{-t/K} = I(1-e^{-t/K}) \tag{5}$$

Combining Equation 4 and Equation 5 gives the final solution for $Q(t)$:

$$Q(t) = I(1-e^{-t/K}) + Q_0 e^{-t/K} \tag{6}$$

E.g. for $Q_0 = 0.1$ m^3/h and $K = 7$ h, we can plot the outflow $Q(t)$ for different constant inflows I. In the examples that follow, we'll use m^3/h and h for the units instead of the standard SI-units for convenience. Define the analytical solutions as a function:

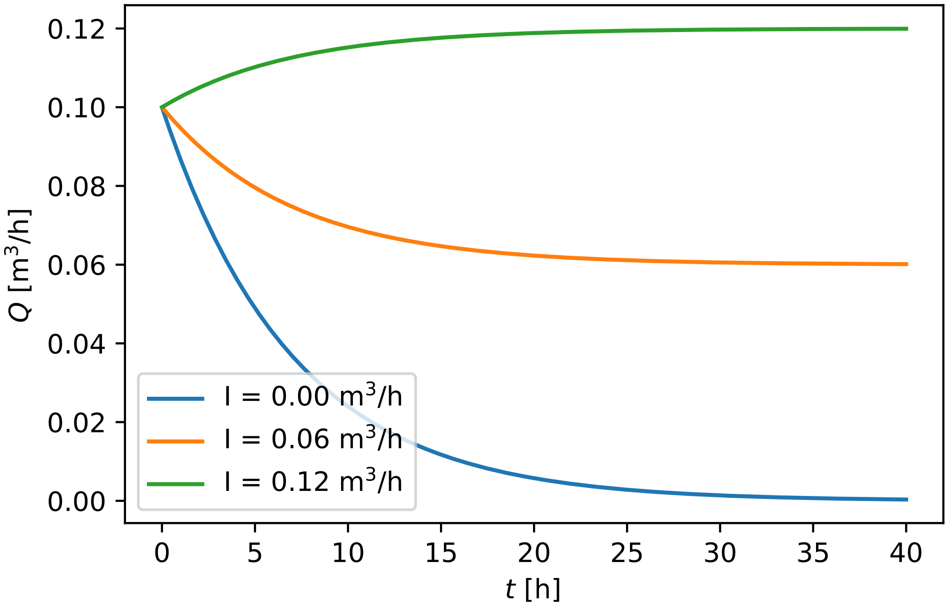
```python
# Imports
import matplotlib.pyplot as plt
import numpy as np
import sympy as sp
import scipy


def Q_analytical_linear(t, I, K, Q0):
    return I * (1 - np.exp(-t / K)) + Q0 * np.exp(-t / K)
```

define the parameters and the three different inflows:

```python
Q_0_num = 0.1  # m^3/h
K_num = 7  # h
t_array = np.linspace(0.01, 40, 200)  # h
I_array = [0.0, 0.06, 0.12]  # m^3/h
```

to then plot the results:

```python
fig, ax = plt.subplots()
colors = plt.rcParams["axes.prop_cycle"].by_key()["color"]
for i, I_ in enumerate(I_array):
    Q_ = Q_analytical_linear(t_array, I_, K_num, Q_0_num)
    ax.plot(t_array, Q_, color=colors[i], label=rf"I = {I_:.2f} m$^3$/h")
ax.set_xlabel(r"$t$ [h]")
ax.set_ylabel(r"$Q$ [m$^3$/h]")
ax.legend()
```



Additionally, note that we can automate the algebraic routines above using SymPy. First define all symbols:

```
K, I, Q0 = sp.symbols("K I Q0", positive=True)
t = sp.symbols("t", positive=True)
s = sp.symbols("s")
q, Q = sp.symbols("q, Q", cls=sp.Function)
```

and then define the ODE in the form $K\frac{dQ}{dt} - (I - Q) = 0$:

```
ode_linear = K * sp.diff(Q(t), t) - (I - Q(t))
ode_linear
```

$$-I + K\frac{d}{dt}Q(t) + Q(t)$$

Now take the Laplace transform of the ODE defining $q(s) = \mathcal{L}\{Q(t)\}$:

```
ode_linear_laplace = sp.laplace_transform(ode_linear, t, s, noconds=True)
ode_linear_laplace = sp.laplace_correspondence(ode_linear_laplace, {Q: q})
ode_linear_laplace
```

$$-\frac{I}{s} + K\left(sq(s) - Q(0)\right) + q(s)$$

We now insert the initial condition $Q(0) = Q_0$:

```
ode_linear_laplace_ic = sp.laplace_initial_conds(ode_linear_laplace, t, {Q:
↪  [Q0]})
ode_linear_laplace_ic
```

$$-\frac{I}{s}+K\left(-Q_0+sq(s)\right)+q(s)$$

In the equation above, the only unknown is $q(s)$, which can be solved for:

```
q_s_solved = sp.solve(ode_linear_laplace_ic, q(s))[0]
q_s_solved
```

$$\frac{I+KQ_0s}{s\left(Ks+1\right)}$$

Now apply the inverse Laplace transform to get $Q(t)$:

```
Q_solved = sp.simplify(sp.inverse_laplace_transform(q_s_solved, s, t))
Q_solved
```

$$\left(Ie^{\frac{t}{K}}-I+Q_0\right)e^{-\frac{t}{K}}$$

Which is identical to the analytical solution Equation 6 derived above.

1.1.2 Second order linear reservoir with constant input

We now get two reservoirs in series:

$$\begin{aligned} K^{(1)}\frac{dQ^{(1)}}{dt} &= I-Q^{(1)} \\ K^{(2)}\frac{dQ^{(2)}}{dt} &= Q^{(1)}-Q^{(2)} \end{aligned}$$

The same SymPy approach can be extended to solve this coupled system. To simplify the solution, we'll assume:

- The second reservoir is initially empty, i.e. $Q^{(2)}(0)=0$.
- The first reservoir has an initial outflow $Q^{(1)}(0)=Q_0$ as it is assumed non-empty.
- The first and second reservoirs have the same residence time, i.e. $K^{(1)}=K^{(2)}=K$.

We define two unknown functions and their Laplace transforms:

```
q1, q2, Q1, Q2 = sp.symbols("q1 q2 Q1 Q2", cls=sp.Function)
```

Write both ODEs in the form $(\dots) = 0$:

```
ode1 = K * sp.diff(Q1(t), t) - (I - Q1(t))
ode2 = K * sp.diff(Q2(t), t) - (Q1(t) - Q2(t))
display(ode1, ode2)
```

$$-I + K\frac{d}{dt}Q_1(t) + Q_1(t)$$

$$K\frac{d}{dt}Q_2(t) - Q_1(t) + Q_2(t)$$

Take the Laplace transform using $q_1(s) = \mathcal{L}\{Q_1(t)\}$ and $q_2(s) = \mathcal{L}\{Q_2(t)\}$:

```
ode1_laplace = sp.laplace_transform(ode1, t, s, noconds=True)
ode1_laplace = sp.laplace_correspondence(ode1_laplace, {Q1: q1})

ode2_laplace = sp.laplace_transform(ode2, t, s, noconds=True)
ode2_laplace = sp.laplace_correspondence(ode2_laplace, {Q1: q1, Q2: q2})
```

Insert the initial conditions:

```
ode1_laplace_ic = sp.laplace_initial_conds(ode1_laplace, t, {Q1: [Q0], Q2:
 ↪  [0]})
ode2_laplace_ic = sp.laplace_initial_conds(ode2_laplace, t, {Q1: [Q0], Q2:
 ↪  [0]})
display(ode1_laplace_ic, ode2_laplace_ic)
```

$$-\frac{I}{s} + K\left(-Q_0 + sq_1(s)\right) + q_1(s)$$

$$Ksq_2(s) - q_1(s) + q_2(s)$$

Solve the algebraic system for $q_1(s)$ and $q_2(s)$:

```
q_s_solved_2 = sp.solve([ode1_laplace_ic, ode2_laplace_ic], [q1(s), q2(s)],
 ↪  dict=True)[0]
q2_s = q_s_solved_2[q2(s)]
q2_s
```

$$\frac{I + KQ_0 s}{K^2 s^3 + 2Ks^2 + s}$$

To get a solution closer to the notation of the theory, we'll first break up into partial fractions:

```
q2_s_partial = sp.apart(q2_s, s)
q2_s_partial
```

$$-\frac{IK}{Ks + 1} + \frac{I}{s} - \frac{K\left(I - Q_0\right)}{\left(Ks + 1\right)^2}$$

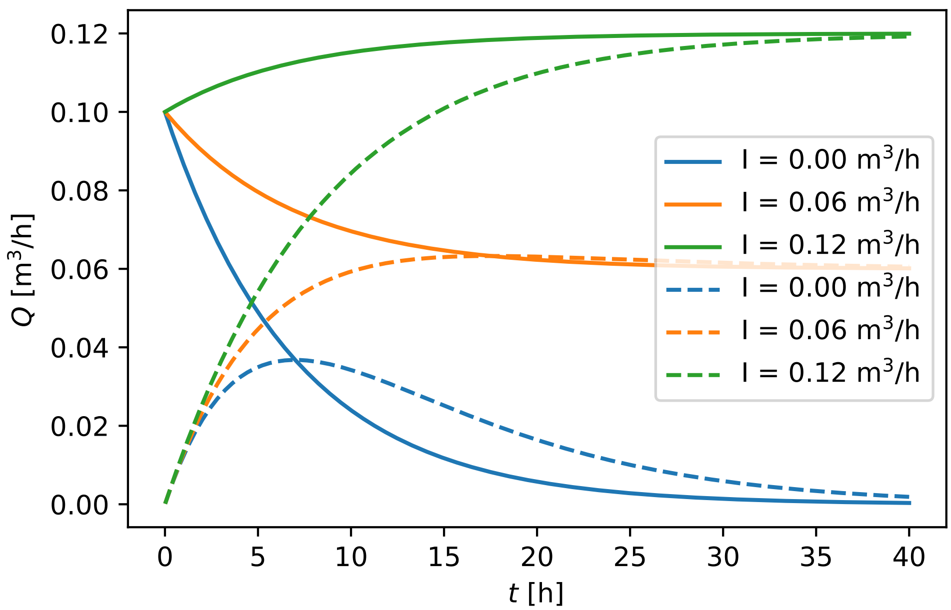
Finally, apply the inverse Laplace transform to obtain $Q_2(t)$.

```
Q2_solved = sp.inverse_laplace_transform(q2_s_partial, s, t)
Q2_solved
```

$$I - Ie^{-\frac{t}{K}} - \frac{t\left(I - Q_0\right)e^{-\frac{t}{K}}}{K}$$

Also $Q_2(t)$ can now be visualised for different constant inflows I and the same parameters as before:

```
for i, I_ in enumerate(I_array):
    Q2_func = sp.lambdify(t, Q2_solved.subs({I: I_, K: K_num, Q0: Q_0_num}),
 ↪  "numpy")
    Q2_ = Q2_func(t_array)
    ax.plot(
        t_array, Q2_, label=rf"I = {I_:.2f} m$^3$/h", color=colors[i],
 ↪  linestyle="--"
    )
ax.set_xlabel(r"$t$ [h]")
ax.set_ylabel(r"$Q$ [m$^3$/h]")
ax.legend()
fig
```



In this figure, full lines are the output of the first reservoir and dashed lines are the output of the second reservoir. Note that the second reservoir has a delayed response compared to the first reservoir.

Given that the second reservoir is initially empty and receives no external input I, Q_2 is entirely defined by Q_1 and $K^{(1)} = K$. So if you solve for Q_2 (in the s-domain $q_2(s)$) with an assumed known $Q_1(t)$ function (in the s-domain $q_1(s)$), you get:

```
q2_s_bis = sp.solve(ode2_laplace_ic, q2(s))[0]
q2_s_bis
```

$$\frac{q_1(s)}{Ks+1}$$

This concept can be generalised in what is called a **transfer function**, which is defined as the ratio of the Laplace transforms of the output and input of a system. In this case, the transfer function $H(s)$ is defined as:

$$H(s) = \frac{q_2(s)}{q_1(s)} = \frac{1}{1+Ks} \tag{7}$$

1.1.3 n-th order linear reservoir with constant input

The concept of 2 reservoirs can be generalised to n reservoirs in series, with the general form of the ODEs given by:

$$\frac{dS^{(i)}}{dt} = K\frac{dQ^{(i)}}{dt} = (Q^{(i-1)} - Q^{(i)}), \quad i = 1, 2, \ldots, n$$

Again, we assume $K^{(i)} = K$ for all reservoirs, $Q^{(0)}(t) = I$ and $Q^{(i)}(t = 0) = 0 \quad \forall i > 1$. The easiest way to solve this equation is to use the transfer function concept. The first reservoir is a special case, as it has an external input I and an initial outflow $Q^{(1)}(0) = Q_0$. In the s-domain, $q^{(1)}(s)$ is given by Equation 3. $\forall i > 1$, the transfer function is given by a form analogous to Equation 7:

$$H^{(i)}(s) = \frac{q_i(s)}{q_{i-1}(s)} = \frac{1}{1 + Ks} \tag{8}$$

Consequently, $q^{(n)}(s)$ can be expressed as:

$$q^{(n)}(s) = q^{(1)}(s)\prod_{i=2}^{n} H^{(i)}(s) = \frac{q^{(1)}(s)}{(1 + Ks)^{n-1}} \tag{9}$$

In SymPy, this can be expressed as (reusing the previously solved $q^{(1)}(s)$ denoted as `q_s_solved`):

```
n = sp.symbols("n", positive=True, integer=True)
q_n_s = q_s_solved / ((1 + K * s) ** (n - 1))
q_n_s = sp.refine(q_n_s, sp.Q.gt(n, 1))
display(q_n_s)
```

$$\frac{(I + KQ_0 s)\,(Ks + 1)^{1-n}}{s\,(Ks + 1)}$$

The inverse Laplace transform can be applied to get $Q^{(n)}(t)$:

```
Q_n_solved = sp.refine(sp.inverse_laplace_transform(q_n_s, s, t), sp.Q.gt(n,
 ↪  1))
Q_n_solved
```

$$I\mathcal{L}_s^{-1}\left[\frac{(Ks + 1)^{1-n}}{Ks^2 + s}\right](t) + KQ_0\frac{\partial}{\partial t}\mathcal{L}_s^{-1}\left[\frac{(Ks + 1)^{1-n}}{Ks^2 + s}\right](t)$$

Unfortunately, we run into a limitation of SymPy here, as it can't generalise the inverse Laplace transform for arbitrary n (try calling `sp.simplify` on `Q_n_solved`, it will crash). For specific values of n, we can still get the solutions

```python
n_num_list = [5, 6, 7]
for n_num in n_num_list:
    print(f"n = {n_num}")
    Q_n_solved_specific = sp.simplify(Q_n_solved.subs(n, n_num))
    display(Q_n_solved_specific)
```

```
n = 5
```

$$I\left(1 - e^{-\frac{t}{K}} - \frac{te^{-\frac{t}{K}}}{K} - \frac{t^2e^{-\frac{t}{K}}}{2K^2} - \frac{t^3e^{-\frac{t}{K}}}{6K^3} - \frac{t^4e^{-\frac{t}{K}}}{24K^4}\right) + \frac{Q_0t^4e^{-\frac{t}{K}}}{24K^4}$$

```
n = 6
```

$$I\left(1 - e^{-\frac{t}{K}} - \frac{te^{-\frac{t}{K}}}{K} - \frac{t^2e^{-\frac{t}{K}}}{2K^2} - \frac{t^3e^{-\frac{t}{K}}}{6K^3} - \frac{t^4e^{-\frac{t}{K}}}{24K^4} - \frac{t^5e^{-\frac{t}{K}}}{120K^5}\right) + \frac{Q_0t^5e^{-\frac{t}{K}}}{120K^5}$$

```
n = 7
```

$$I\left(1 - e^{-\frac{t}{K}} - \frac{te^{-\frac{t}{K}}}{K} - \frac{t^2e^{-\frac{t}{K}}}{2K^2} - \frac{t^3e^{-\frac{t}{K}}}{6K^3} - \frac{t^4e^{-\frac{t}{K}}}{24K^4} - \frac{t^5e^{-\frac{t}{K}}}{120K^5} - \frac{t^6e^{-\frac{t}{K}}}{720K^6}\right) + \frac{Q_0t^6e^{-\frac{t}{K}}}{720K^6}$$

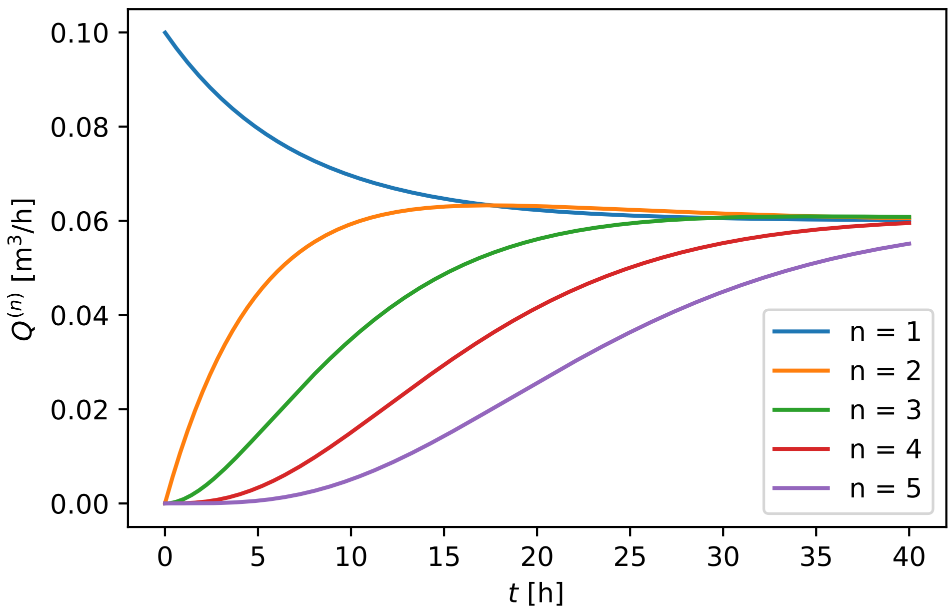
From these solutions, we can easily identify the general pattern that can be expressed as:

$$Q^{(n)}(t) = I - Ie^{-t/K}\sum_{i=0}^{n-1}\frac{(t/K)^i}{i!} + Q_0e^{-t/K}\sum_{i=0}^{n-1}\frac{(t/K)^i}{i!}$$

which is identical to the formulation in the theory notes. We can now also easily plot results for different n values

```python
I_num = 0.06  # m^3/h
n_num_list = [1, 2, 3, 4, 5]
fig, ax = plt.subplots()
for n_num in n_num_list:
    Q_n_solved_specific_ = sp.simplify(Q_n_solved.subs(n, n_num))
    Q_n_func_ = sp.lambdify(
        t, Q_n_solved_specific_.subs({I: I_num, K: K_num, Q0: Q_0_num}),
 ↪  "numpy"
    )
    Q_n_ = Q_n_func_(t_array)
    ax.plot(t_array, Q_n_, label=rf"n = {n_num}")
```

```
ax.set_xlabel(r"$t$ [h]")
ax.set_ylabel(r"$Q^{(n)}$ [m$^3$/h]")
ax.legend()
```



The larger n, the more delayed the response of the outflow is.

1.1.4 Nash-casacde

A Nash-cascade is a special case of the n-th order linear reservoir, where the input I to the first reservoir is a Dirac delta function $\delta(t)$:

$$\delta(t) = \begin{cases} \infty, & t = 0 \\ 0, & t \neq 0 \end{cases}$$

with

$$\int_{-\infty}^{\infty} \delta(t) dt = 1$$

It has the very useful property that $\mathcal{L}\{\delta(t)\} = 1$[1]. Additionally assuming that the first reservoir is initially empty, i.e. $Q^{(1)}(0) = 0$, the continuity equation for the first reservoir of Equation 2 sets $I = \delta(t)$ and $Q(t = 0) = 0$. The ODE can be laplace transformed:

[1]Note that t needs to be defined between $-\infty$ and ∞ for this to be true

```
t = sp.symbols("t")  # Generalise t for correct dirac delta transform!
I_dirac = sp.DiracDelta(t)
ode_linear_nash_1 = K * sp.diff(Q(t), t) - (I_dirac - Q(t))
ode_linear_nash_1_laplace = sp.laplace_transform(ode_linear_nash_1, t, s,
 ↪  noconds=True)
ode_linear_nash_1_laplace =
 ↪  sp.laplace_correspondence(ode_linear_nash_1_laplace, {Q: q})
display(ode_linear_nash_1_laplace)
```

$K\left(sq(s) - Q(0)\right) + q(s) - 1$

where we can clearly see that the Laplace transform of the Dirac delta function is 1. Inserting $Q(0) = 0$ gives and solving for $q(s)$ gives:

```
ode_linear_nash_1_laplace_ic = sp.laplace_initial_conds(
    ode_linear_nash_1_laplace, t, {Q: [0]}
)
q_1_nash = sp.solve(ode_linear_nash_1_laplace_ic, q(s))[0]
display(q_1_nash)
```

$\frac{1}{Ks + 1}$

So in the case of a dirac delta input, also the first reservoir has a transfer function of the same form as the other reservoirs (see Equation 8).

$$H^{(1)}(s) = \frac{q_1(s)}{I(s)} = \frac{q_1(s)}{1} = \frac{1}{1 + Ks}$$

Therefore, Equation 9 can be simplified to:

$$q^{(n)}(s) = \prod_{i=1}^{n} H^{(i)}(s) = \frac{1}{(1 + Ks)^n}$$

Transforming back to the time domain gives:

```
q_n_s_nash = 1 / ((1 + K * s) ** n)
Q_n_solved_nash = sp.simplify(sp.inverse_laplace_transform(q_n_s_nash, s, t))
Q_n_solved_nash
```

$$\frac{K^{-n} t^{n-1} e^{-\frac{t}{K}} \theta\left(t\right)}{\left(n - 1\right)!}$$

In the above equation, $\theta(t)$ is the Heaviside function (cf. supra for definition). As we are only interested in $t \geq 0$ (i.e. after the input is applied) we can simplify the solution to:

```
Q_n_solved_nash_simplified = Q_n_solved_nash.subs(
    sp.Heaviside(t), sp.Heaviside(t).rewrite(sp.Piecewise)
)
Q_n_solved_nash_simplified = sp.refine(Q_n_solved_nash_simplified,
 ↪  sp.Q.positive(t))
Q_n_solved_nash_simplified
```

$$\frac{K^{-n} t^{n-1} e^{-\frac{t}{K}}}{\left(n - 1\right)!}$$

Again note the equivalence to the formulation in the theory notes. Interestingly, the solution here assumes an empty first reservoir and a Dirac delta input, while the theory notes assume no input and a first and $Q^{(1)}(t = 0) = 1$. The two assumptions are equivalent, as the Dirac delta input can be interpreted as a unit input to an empty reservoir.

Because the Nash-cascade is the response to an instantaneous input, it can be interpreted as an Instantaneous Unit Hydrograph (IUH)! Also note that the assumption that n is an integer can be relaxed to n being a positive real number. In this case, applying the inverse Laplace transform yields:

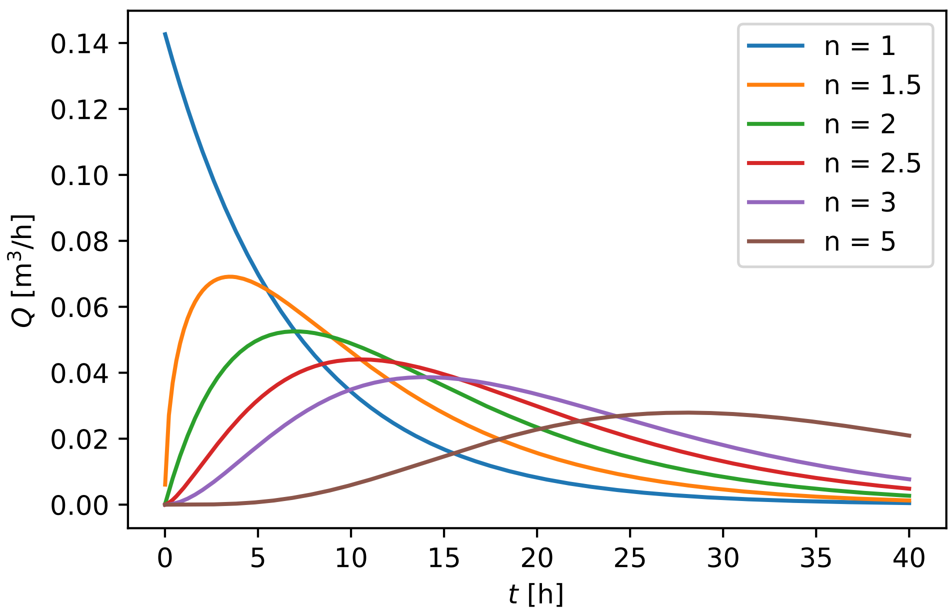
```
n = sp.symbols("n", positive=True, real=True)
q_n_s_nash = 1 / ((1 + K * s) ** n)
Q_n_solved_nash = sp.simplify(sp.inverse_laplace_transform(q_n_s_nash, s, t))
Q_n_solved_nash
```

$$\frac{K^{-n} t^{n-1} e^{-\frac{t}{K}} \theta\left(t\right)}{\Gamma\left(n\right)}$$

This allows n to be calibrated as a real number to match the geomorphological characteristics of a catchment. For completeness, we visualise the Nash-cascade for different n values, including non-integer values:

```
n_num_list = [1, 1.5, 2, 2.5, 3, 5]
fig, ax = plt.subplots()
for n_num in n_num_list:
    Q_n_solved_specific_ = sp.simplify(Q_n_solved_nash.subs(n, n_num))
    Q_n_func_ = sp.lambdify(t, Q_n_solved_specific_.subs({K: K_num}),
 ↪  "numpy")
```

```python
    Q_n_ = Q_n_func_(t_array)
    ax.plot(t_array, Q_n_, label=rf"n = {n_num}")
ax.set_xlabel(r"$t$ [h]")
ax.set_ylabel(r"$Q$ [m$^3$/h]")
ax.tick_params(axis="y")
ax.legend()
```



Application: routing via convolution

Defining $u(t) = \frac{1}{(n-1)!K}(\frac{t}{K})^{n-1} \exp{(-t/K)}$ following the nash cascade, the outflow $Q(t)$ is given by the convolution of $u(t)$ with the input $I(t)$:

$$Q(t) = (u * I)(t) = \int_0^t u(\tau)I(t-\tau)d\tau$$

Define a toy input $I(t)$:

```python
I_array = [0.2, 0.5, 0.1, 0.2, 0.6, 0.8, 1.2, 1, 0.5, 0.2]  # mm/h
A_example = 10_000  # m^2, example catchment area
I_array_m3h = np.array(I_array) * A_example / 1000  # m^3/h
t_array = np.arange(len(I_array))  # h
```

where each value represent the average rainfall intensity over a 1h time step. We can make a piecewise constant function of this input:

```python
I_func = scipy.interpolate.interp1d(
    t_array, I_array_m3h, kind="previous", fill_value=0,
↪   bounds_error=False
)
t_eval = np.linspace(0, 60, 61) # h
```

Now wen can numerically integrate the convolution with `scipy.integrate.quad` to get $Q(t)$ for different n values:

```python
u_nash_func = sp.lambdify((t, n, K), Q_n_solved_nash, "numpy")


def integrand_func(tau, t, n, K):
    return I_func(tau) * u_nash_func(t - tau, n, K)


Q_numerical_dict = {}
for n_num in n_num_list:
    print("Integrating for n =", n_num)
    Q_numerical_ = np.zeros_like(t_eval)
    for i, t_ in enumerate(t_eval):
        Q_numerical_[i] = scipy.integrate.quad(
            integrand_func, 0, t_, args=(t_, n_num, K_num),
↪   points=t_array[t_array <= t_]
        )[0] # Use points argument to handle discontinuities in I_func
    Q_numerical_dict[n_num] = Q_numerical_
```

```
Integrating for n = 1
Integrating for n = 1.5
Integrating for n = 2
Integrating for n = 2.5
Integrating for n = 3
Integrating for n = 5
```

Now we can plot the input and output:

```python
# For plotting rainfall take higher resolution timeseries
t_eval_hr = np.linspace(0, 60, 1000)  # h
I_hr = I_func(t_eval_hr)

fig, ax = plt.subplots()
for n_num, Q_numerical_ in Q_numerical_dict.items():
    ax.plot(t_eval, Q_numerical_, label=rf"n = {n_num}", zorder=3)
ax2 = ax.twinx()
ax2.fill_between(t_eval_hr, I_hr, color="blue", label="Rainfall",
 ↪  alpha=0.8, zorder=2)
ax2.set_ylim(0, np.max(I_hr) * 5)
ax2.set_ylabel(r"$I$ [m$^3$/h]", color="blue")
ax2.tick_params(axis="y", colors="blue")
ax2.invert_yaxis()
ax.set_xlabel(r"$t$ [h]")
ax.set_ylabel(r"$Q$ [m$^3$/h]")
ax.legend()
```